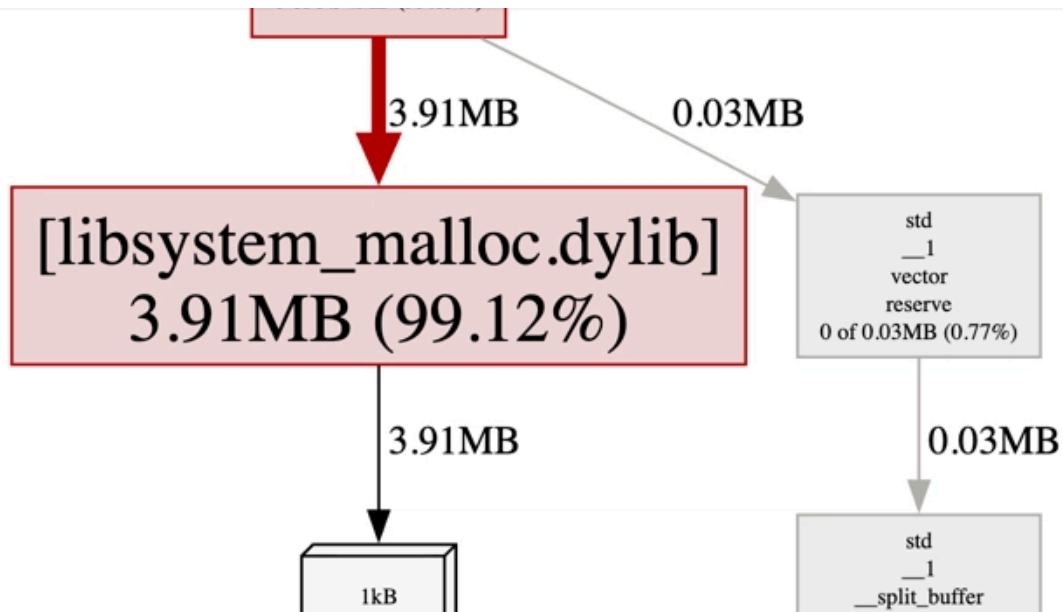




Frank Rosner

Posted on Dec 1, 2025



11



1



1



1



1

libmalloc, jemalloc, tcmalloc, mimalloc - Exploring Different Memory Allocators

#cpp #benchmarking #memory #performance

What Are Memory Allocators?

Applications need memory to store data and code during runtime. Memory can be allocated statically (fixed size at compile time), or dynamically (at runtime). Dynamic memory allocation is crucial when the memory size needed varies during program execution, which is the case for most modern applications.

The stack and the heap are two key memory regions during program execution. Stack allocation is used for function calls and local variables and it happens automatically. The stack allocation lifecycle is tied to the lifecycle of the function. The heap is used for dynamic memory allocation. In non-garbage-collected languages like C++, the programmer is responsible for managing the heap, while in garbage-collected languages like Java, the heap is managed automatically.

In many cases, heap allocation and de-allocation is implemented via a memory allocator, which implements functions like `malloc` and `free`. Those generic

implementation.

In [Writing My Own Dynamic Memory Management](#) I attempted to write a very simple allocator for my own operating system based on a doubly linked list. The following animation shows how the available heap is managed by the allocator:

Dynamic Memory Allocation using a Doubly Linked List

Frank Rosner



Watch on

Efficient memory allocation is a complex problem, especially on modern computer architectures. Modern allocators combine advanced data structures and algorithms to achieve high performance in concurrent environments.

Especially in performance critical applications, such as databases, web servers, and game engines, the choice of memory allocator can have a significant impact on performance. I wanted to learn more about the different allocators available. In this blog post we are going to compare a few well-known allocators on MacOS:

- [libmalloc](#) - The default allocator on MacOS, developed by Apple.
- [jemalloc](#) - Created by Jason Evans originally for FreeBSD to address fragmentation and scaling issues, jemalloc is a scalable allocator widely adopted in performance-critical applications including Firefox and Facebook.
- [tcmalloc](#) - Developed by Google as part of the [Google Performance Tools](#) to enhance multithreaded allocation speed and reduce lock contention using thread-local or core-local caches.

local free lists and free list sharding for performance gains.

- [hoard](#) - Designed by Emery Berger and his team at the University of Massachusetts to reduce memory fragmentation and contention in multithreaded systems by partitioning heaps per thread, introduced in the early 2000s as a research-driven allocator.

Allocator Architectures

All modern memory allocators share common architectural concepts to manage dynamic memory efficiently and safely. Allocation requests can come in different sizes, ranging from a few bytes to megabytes or even gigabytes. Allocators need to be equipped with strategies to handle different allocation sizes with minimal overhead and fragmentation. Commonly this is achieved by using some form of segmentation based on the requested size.

Allocators need to track the state of allocated and free memory. This is often done by using data structures that keeps track of the state of each memory block. Metadata can be tracked externally, in a separate data structure, or internally within the block, or a combination of both.

In multithreaded environments, concurrency control is necessary to ensure safety when allocating and deallocating memory. Synchronization negatively impacts performance, however, so modern allocators use various techniques to minimize synchronization overhead, e.g. by using thread-local data structures and even entire heap regions. Of course, these techniques come with additional memory overhead.

Benchmarks

What to Compare?

While the interface looks simple, the implementations of those allocators differ significantly. Different allocators have different performance characteristics, and are better suited for different workloads and computer architectures. When comparing allocators, there are several key performance indicators (KPIs) to consider:

- **Throughput** (ops/sec)
- **Latency** - (sec/op)
- **Memory usage** - (overhead and fragmentation)
- **Tooling and Usability** (debugging, profiling, leak checking, ...)

The workload (allocation size, frequency, number of threads, etc.) impacts these KPIs, so it is important to benchmark your specific workload. While there are also platform restrictions that might limit your choice (e.g. `libmalloc` is only available on Apple operating systems), I will ignore the platform limitations in the comparison.

Benchmarking Setup

I'm running the benchmarks using Google Benchmark v1.9.4 on my Nov 2023 MacBook Pro (M3) with MacOS 15.6.1, compiled with Apple `clang-1700.0.13.5`. You can find the [source code](#) on GitHub.

While `libmalloc` is the default allocator on MacOS and part of `libSystem`, the other allocators are going to be installed via `brew`. Note that `tcmalloc` is part of `gperftools`, and `libhoard` is a custom tap (`brew tap emeryberger/hoard`). Here are the versions I am using:

```
# otool -L build/malloc-post-benchmark-libmalloc
/usr/lib/libSystem.B.dylib (current version 1351.0.0)

# brew info jemalloc | grep Cellar
/opt/homebrew/Cellar/jemalloc/5.3.0

# brew info gperftools | grep Cellar
/opt/homebrew/Cellar/gperftools/2.17.2

# brew info mimalloc | grep Cellar
/opt/homebrew/Cellar/mimalloc/3.1.5

# brew info emeryberger/hoard/libhoard | grep Cellar
/opt/homebrew/Cellar/libhoard/HEAD-5a7073f
```

I am using CMake to build the benchmark binaries for each allocator. The gist of the `CMakeLists.txt` is:

```
set(MALLOC_IMPLEMENTATIONS jemalloc mimalloc hoard tcmalloc)
foreach(MALLOC ${MALLOC_IMPLEMENTATIONS})
    find_library(${MALLOC}_LIBRARY ${MALLOC})
    set(EXE_NAME "${PROJECT_NAME}-benchmark-${MALLOC}")
    add_executable(${EXE_NAME} src/main.cpp)
    target_link_libraries(${EXE_NAME} PRIVATE benchmark::benchmark pthread)
    target_link_libraries(${EXE_NAME} PRIVATE ${${MALLOC}_LIBRARY})
endforeach()
```

benchmarks in a loop. Thanks to the `--benchmark_filter` command line option and the way Google Benchmark builds benchmark names, we can loop over different parameters for a given benchmark, restarting the binary after each run.

```
run_allocation_throughput_benchmark() {
    local size=$1
    local threads=$2
    echo "Running allocation throughput benchmark for ${MALLOC} with ${size} si
    ${executable} --benchmark_filter="BM_AllocationThroughput/${size}/iteration
    --benchmark_out="results/${MALLOC}_AllocationThroughput_${size}_${threads}
    > /dev/null
}

executable_prefix="./build/malloc-post-benchmark-"

for executable in ${executable_prefix}*; do
    MALLOC="${executable#./build/malloc-post-benchmark-}"
    for threads in 1 2 4 8; do
        for size in {1..22}; do
            run_allocation_throughput_benchmark $((2**size)) ${threads}
        done
    done
done
```



We are storing the results in JSON files, which we combine, analyze and visualize using [matplotlib](#) in Python. Here's the structure of a benchmark result file:

```
{
  "context": {
    "date": "2025-11-26T14:00:48+01:00",
    "host_name": "MyMacBook",
    "executable": "./build/malloc-post-benchmark-hoard",
    "num_cpus": 12,
    "mhz_per_cpu": 24,
    "cpu_scaling_enabled": false,
    "caches": [
      {
        "type": "Data",
        "level": 1,
        "size": 65536,
        "num_sharing": 0
      },
      {
```

```

        "num_sharing": 0
    },
    {
        "type": "Unified",
        "level": 2,
        "size": 4194304,
        "num_sharing": 1
    }
],
"load_avg": [2.55762, 2.97754, 3.83203],
"library_version": "v1.9.4",
"library_build_type": "debug",
"json_schema_version": 1
},
"benchmarks": [
    {
        "name": "BM_AllocationThroughput/2/iterations:1000/threads:1",
        "family_index": 0,
        "per_family_instance_index": 0,
        "run_name": "BM_AllocationThroughput/2/iterations:1000/threads:1",
        "run_type": "iteration",
        "repetitions": 1,
        "repetition_index": 0,
        "threads": 1,
        "iterations": 1000,
        "real_time": 5.1424708217382431e+04,
        "cpu_time": 5.1425000000000051e+04,
        "time_unit": "ns",
        "items_per_second": 1.9445794846864346e+07
    }
]
}

```

Now with the setup in place, let's look into the different KPIs in greater detail.

Throughput

To measure throughput, we will design a benchmark that within each iteration, allocates memory of a given size for a fixed number of pointers (1000), then frees and reallocates memory for 1000 of these pointers at random, and finally frees all pointers. This yields a total of 2000 memory allocations and frees per iteration. For the throughput counter `SetItemsProcessed` we treat two `malloc` plus two `free` calls as one "item".

```

size_t n = 1000;

std::vector<void*> ptrs(n);

for (auto _ : state) {
    std::mt19937 rng(std::hash<std::thread::id>{}(std::this_thread::get_id()));
    std::uniform_int_distribution<int> dist(0, n - 1);

    for (size_t i = 0; i < n; ++i) {
        ptrs[i] = malloc(sz);
        if (!ptrs[i]) state.SkipWithError("malloc failed");
    }
    benchmark::DoNotOptimize(ptrs);

    for (size_t i = 0; i < n; ++i) {
        int j = dist(rng);
        free(ptrs[j]);
        ptrs[j] = malloc(sz);
    }

    for (size_t i = 0; i < n; ++i) {
        free(ptrs[i]);
    }
}

state.SetItemsProcessed(state.iterations() * n);
}

```



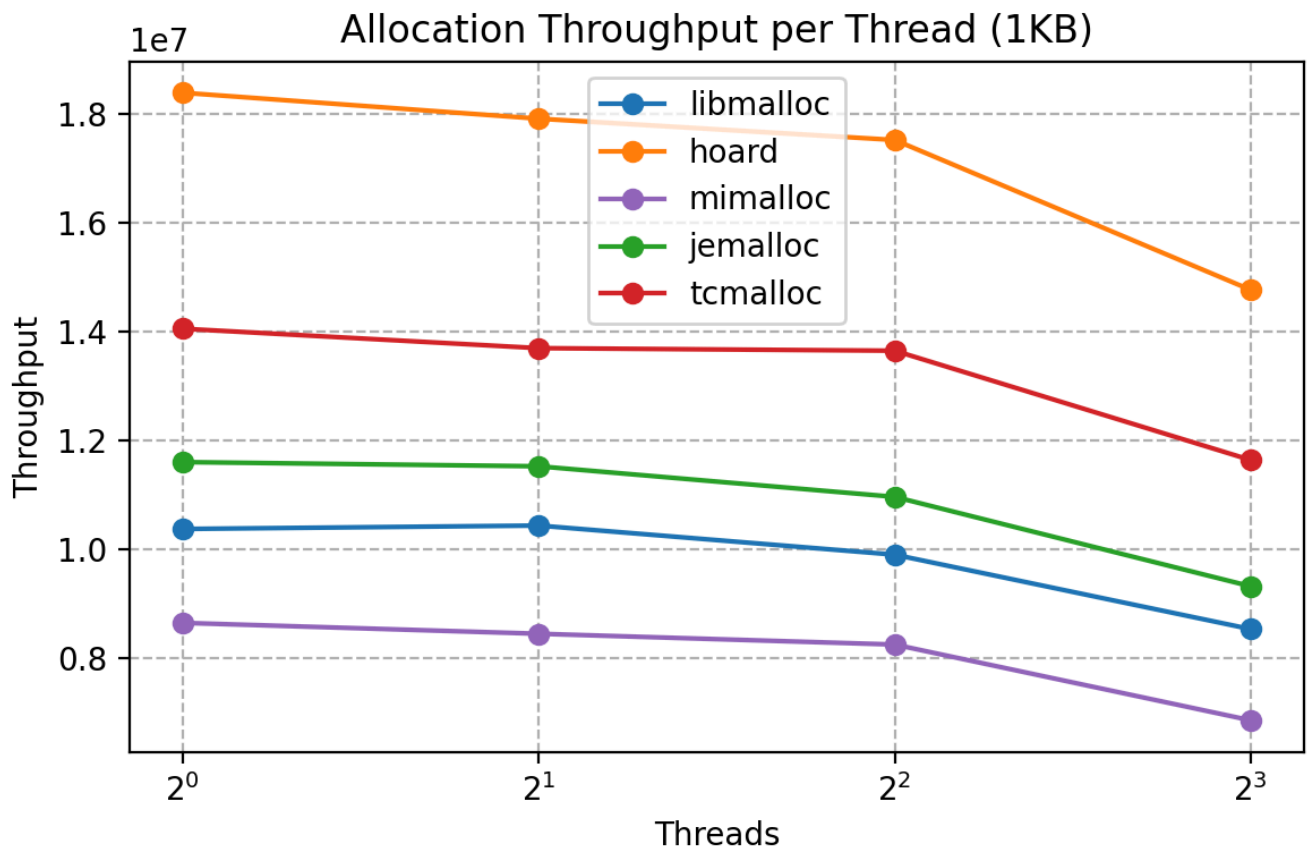
We can then run the benchmark for different allocation sizes and different number of threads:

```

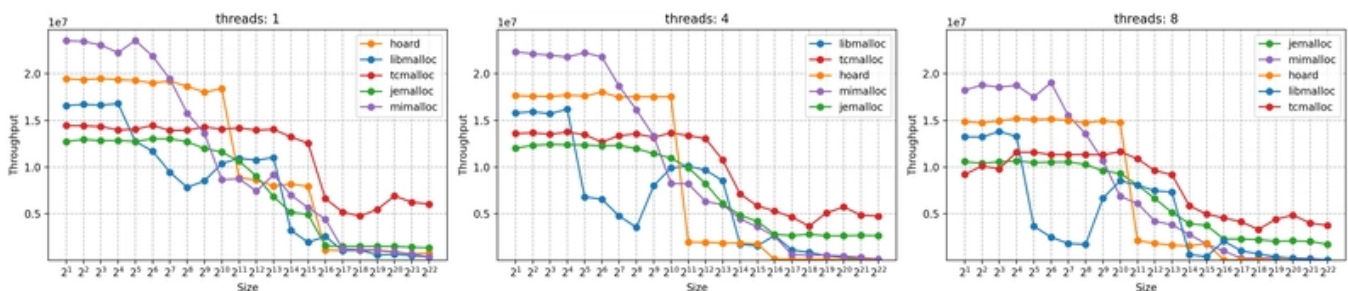
BENCHMARK(BM_AllocationThroughput)
    ->RangeMultiplier(2)
    ->Range(1 << 1, 1 << 25)
    ->Iterations(1000)
    ->Threads(1)
    ->Threads(2)
    ->Threads(4)
    ->Threads(8);

```

We expect the allocation throughput per thread to decrease with increased parallelism due to the increased synchronization overhead. When plotting the

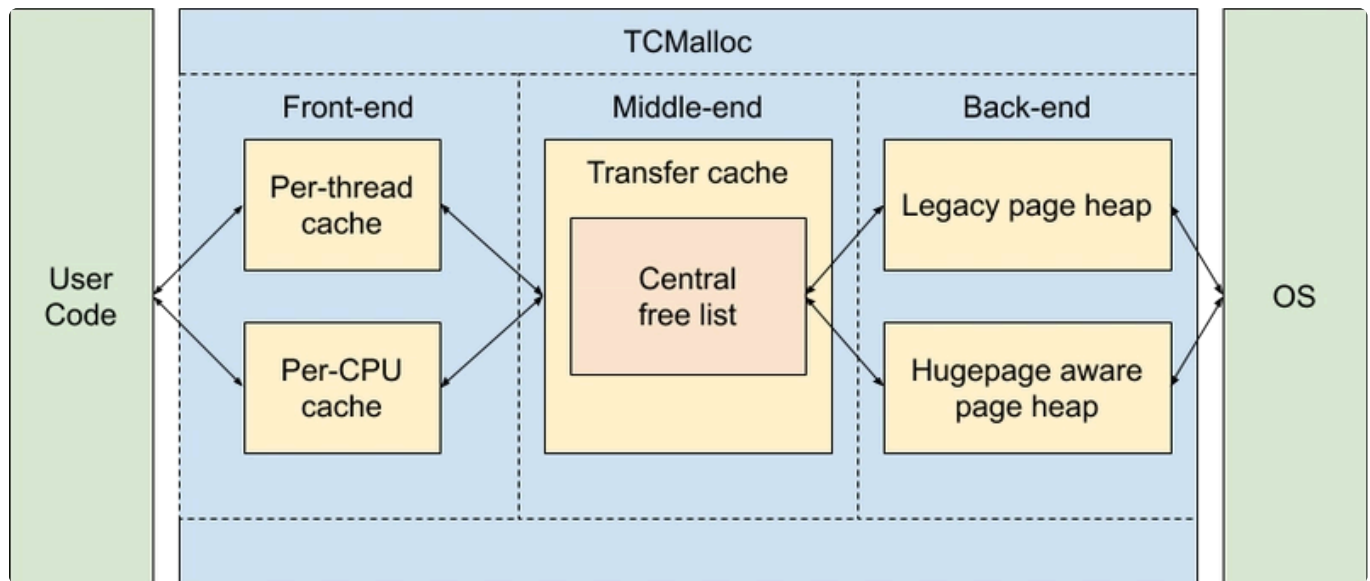


We can also see that `hoard` has the highest throughput, more than 2x of what `mimalloc` achieves. This is only one data point, however, as we were looking at 1KB allocations. Let's look at the throughput for different allocation sizes and different number of threads:



As you can see, the different allocators have vastly different throughput characteristics across the different workloads. While both `hoard` and `mimalloc` perform very well for small allocations, their throughput decreases rapidly for allocations > 1KB. `tcmalloc` takes the lead for allocations > 1KB and maintains a steady throughput up to 32KB (2¹⁵ bytes). `jemalloc` has the lowest throughput for smaller allocation sizes, but maintains a decent throughput especially with increased parallelism compared to `mimalloc`, `hoard`, and `libmalloc`. In very large allocations,

The sharp drop in throughput for larger allocations in the different allocators can be explained by the way they handle them internally. `tcmalloc` for example handles small allocations within the per-CPU caches in the front-end, while larger allocations have to go through the central free list, increasing lock contention (see architecture diagram below, taken from the `tcmalloc` [design documentation](#)). The thresholds depend on the page size and can be viewed in the [size class definitions](#).



Next, let's take a look at the latency of the different allocators.

Latency

For the latency benchmark, I was interested in the latency of the `malloc` call. To measure that, I used manual timing, measuring only the time spent in the `malloc` call:

```
static void BM_AllocationLatency(benchmark::State& state) {
    size_t sz = size_t(state.range(0));

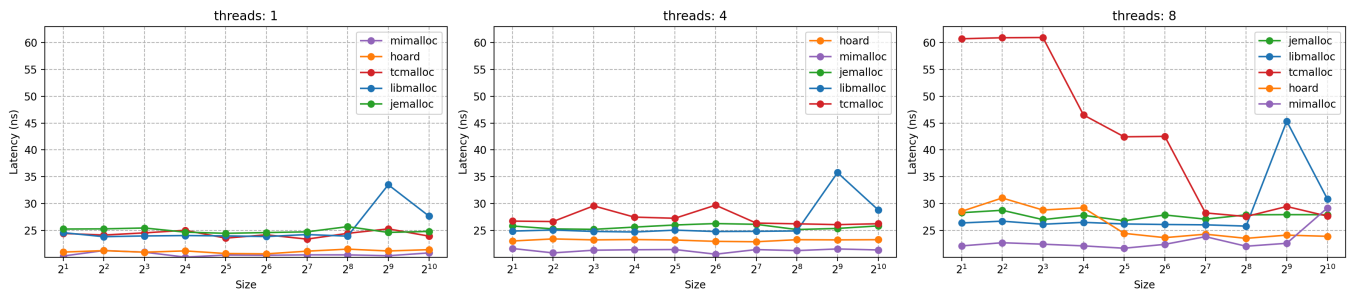
    for (auto _ : state) {
        state.PauseTiming();

        auto start = std::chrono::high_resolution_clock::now();
        void* ptr = malloc(sz);
        auto end = std::chrono::high_resolution_clock::now();

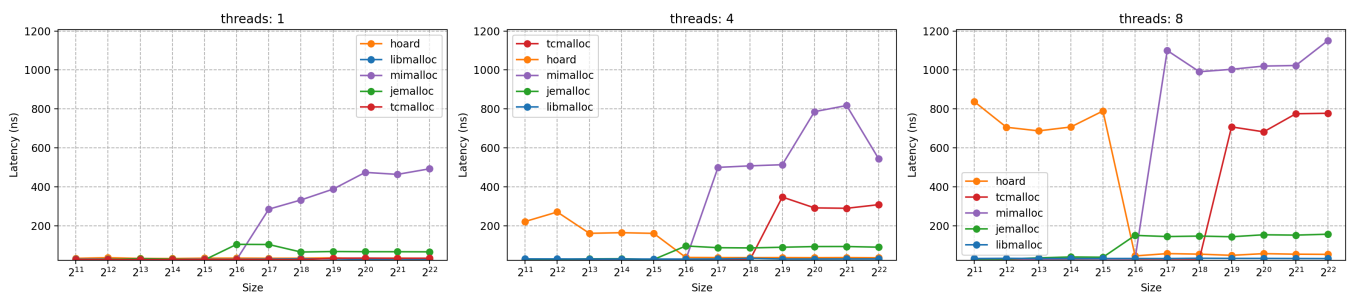
        if (!ptr) state.SkipWithError("malloc failed");

        auto elapsed = std::chrono::duration_cast<std::chrono::duration<double>
state.SetIterationTime(elapsed.count());
    }
}
```

Since the latency difference between small and large allocations is in orders of magnitude, I will plot small ($\leq 1\text{KB}$) and larger ($> 1\text{KB}$) results separately:



We can see that all allocators perform small allocations within 20-30ns, except for `tcmalloc` in the face of a larger amount of threads and very small ($\leq 64\text{B}$) allocation sizes.



For larger allocations in a single-threaded environment, all allocators perform reasonably well. Only `mimalloc` starts to experience a significant latency increase for allocations $> 64\text{KB}$. When multiple threads come into play, `hoard` experiences a significant latency increase for allocations between 2KB and 32KB and `tcmalloc` also starts to see a significant latency increase for allocations $> 262\text{KB}$.

Memory Usage

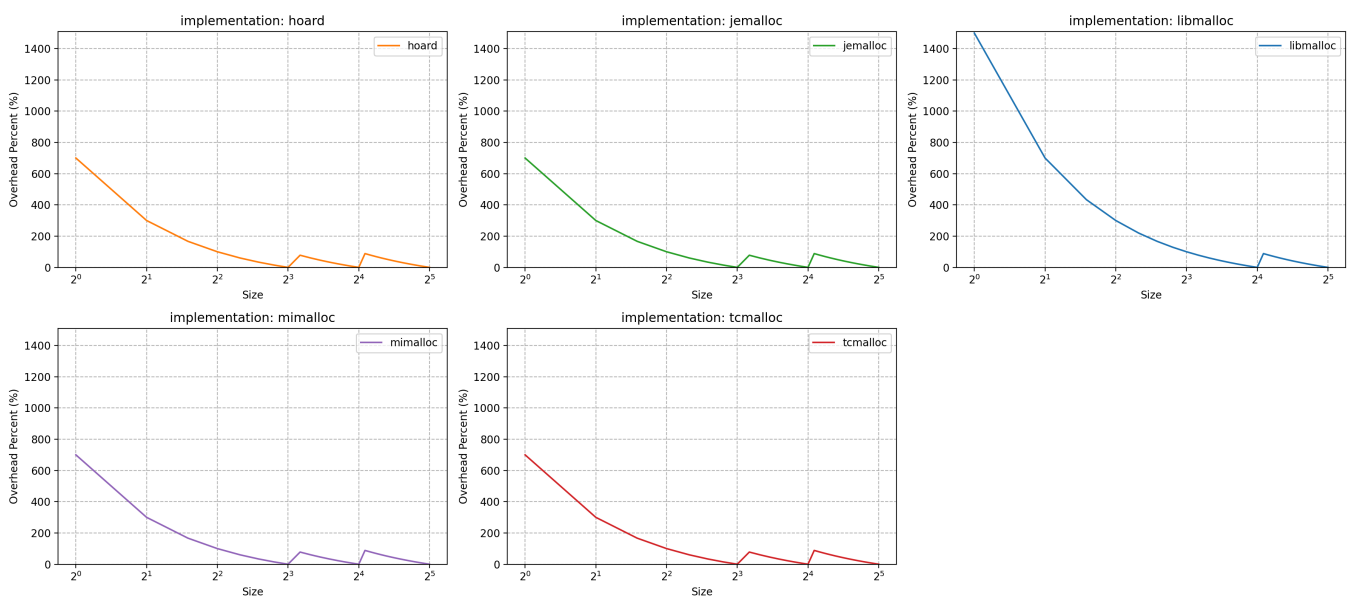
When it comes to memory usage, we mainly care about two types of overhead:

- The allocation overhead when the allocation size is not aligned with the internal page size
- The bookkeeping / synchronization overhead (can be per thread, per core, per pointer)

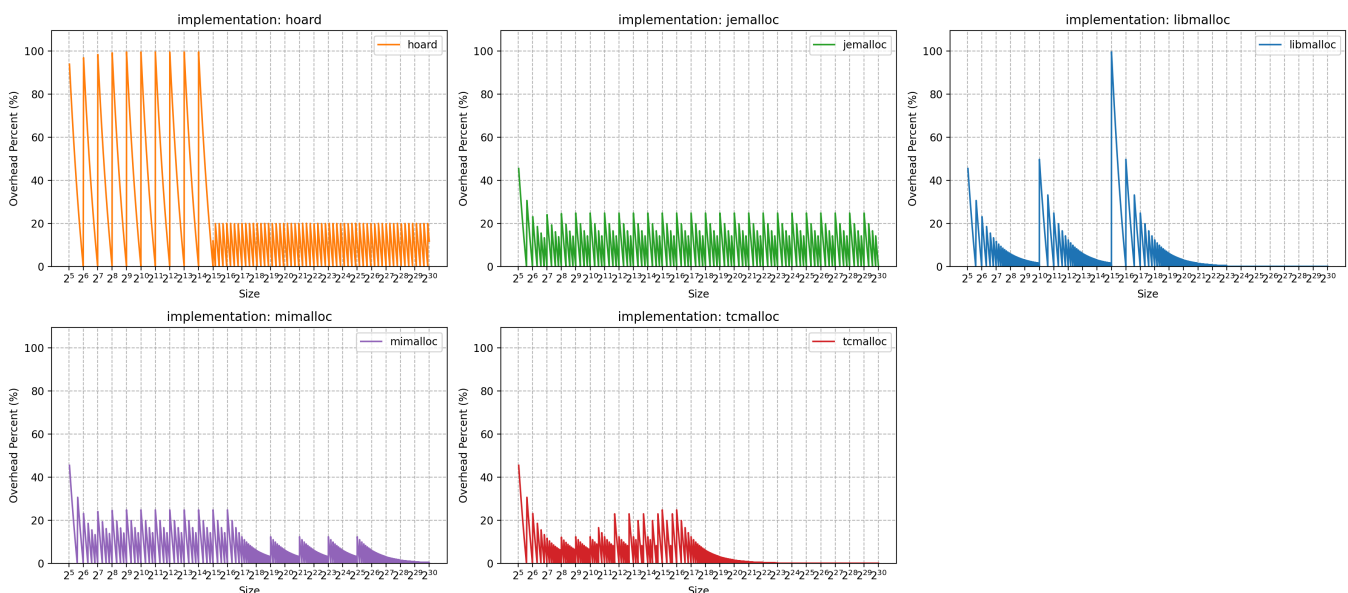
allocation:

```
void* ptr = malloc(sz);
size_t actual = malloc_size(ptr);
size_t overhead = actual - sz;
```

As expected, for tiny allocations, the overhead is very high (up to 1500% when allocating 1B with `libmalloc`). If your application is very memory constrained, `tcmalloc` has a "small-but-slow" mode that can be used when memory footprint minimization is more important than performance.



Starting from 16B, all allocators reach a reasonable overhead $\leq 100\%$. Let's look at the overhead for larger allocations:



allocation sizes, `libmalloc` and `hoard` have a much higher overhead. `hoard` continuously reaches 100% overhead for allocations $\leq 32\text{KB}$. `libmalloc` has peaks at 33B, $\sim 1\text{KB}$, and $32\text{KB}+1\text{B}$.

In practice, to reduce waste, you should aim for allocations that are powers of two or at least aligned with the page size. On MacOS, you can use `malloc_good_size` to get the closest size that will not waste space, but it is not available on Linux. You can also prefer fewer, larger, long-lived buffers over many tiny allocations.

The graphs also reveal information about the internal thresholds related to sizes. E.g. on `libmalloc`, the [zone thresholds](#) align with the peaks in the graph:

- TINY zone handles allocations up to 1008 bytes ($\sim 2^{10}$ bytes).
- SMALL zone handles allocations from above TINY up to 32 KB (2^{15} bytes).
- MEDIUM zone handles allocations from above SMALL up to 8 MB (2^{23} bytes).
- LARGE zone handles allocations beyond the MEDIUM threshold.

In addition to the overhead per allocation, there is also bookkeeping overhead. Since measuring this accurately is more involved, I decided to write a simple program and measure the resident set size (RSS) of the process over time. On MacOS, we can use the Mach API (`mach/mach.h`), specifically the `task_info` function.

```
size_t get_rss_bytes() {
    struct mach_task_basic_info info;
    mach_msg_type_number_t count = MACH_TASK_BASIC_INFO_COUNT;
    if (task_info(mach_task_self(), MACH_TASK_BASIC_INFO,
                 (task_info_t)&info, &count) == KERN_SUCCESS) {
        return info.resident_size;
    }
    return 0;
}
```

Next, we design a "worker" function that we can launch in a thread. It will spin until an external atomic flag indicates it to stop. First it will fill a vector of pointers with allocated memory, filled with 0s. Once all pointers are filled, it randomly selects a pointer to free and reallocate, simulating a workload. Before stopping, it frees all pointers.

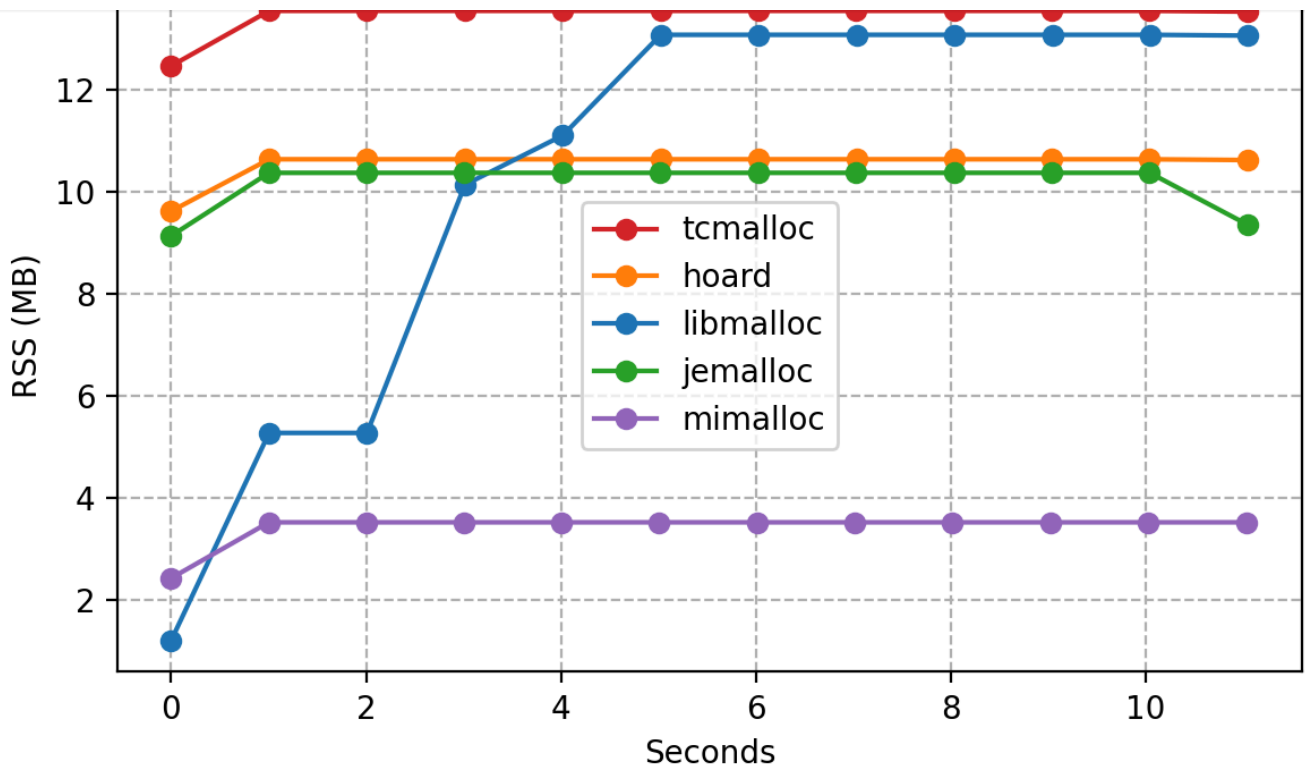
```
std::atomic<bool> should_stop(false);
```

```
while (!should_stop.load()) {
    if (pointers.size() < num_pointers) {
        void* ptr = malloc(pointer_size);
        if (ptr != nullptr) {
            memset(ptr, 0, pointer_size);
            pointers.push_back(ptr);
        }
    } else {
        size_t idx = rand() % pointers.size();
        free(pointers[idx]);
        void* ptr = malloc(pointer_size);
        if (ptr != nullptr) {
            memset(ptr, 0, pointer_size);
            pointers[idx] = ptr;
        }
    }
}

for (void* ptr : pointers) {
    free(ptr);
}
```

In the main method, we can submit this function to a given number of threads, passing a given number of pointers and pointer size. We are going to capture the RSS size in the main thread before launching the threads, every 1 second while the threads are running, and after stopping all threads.

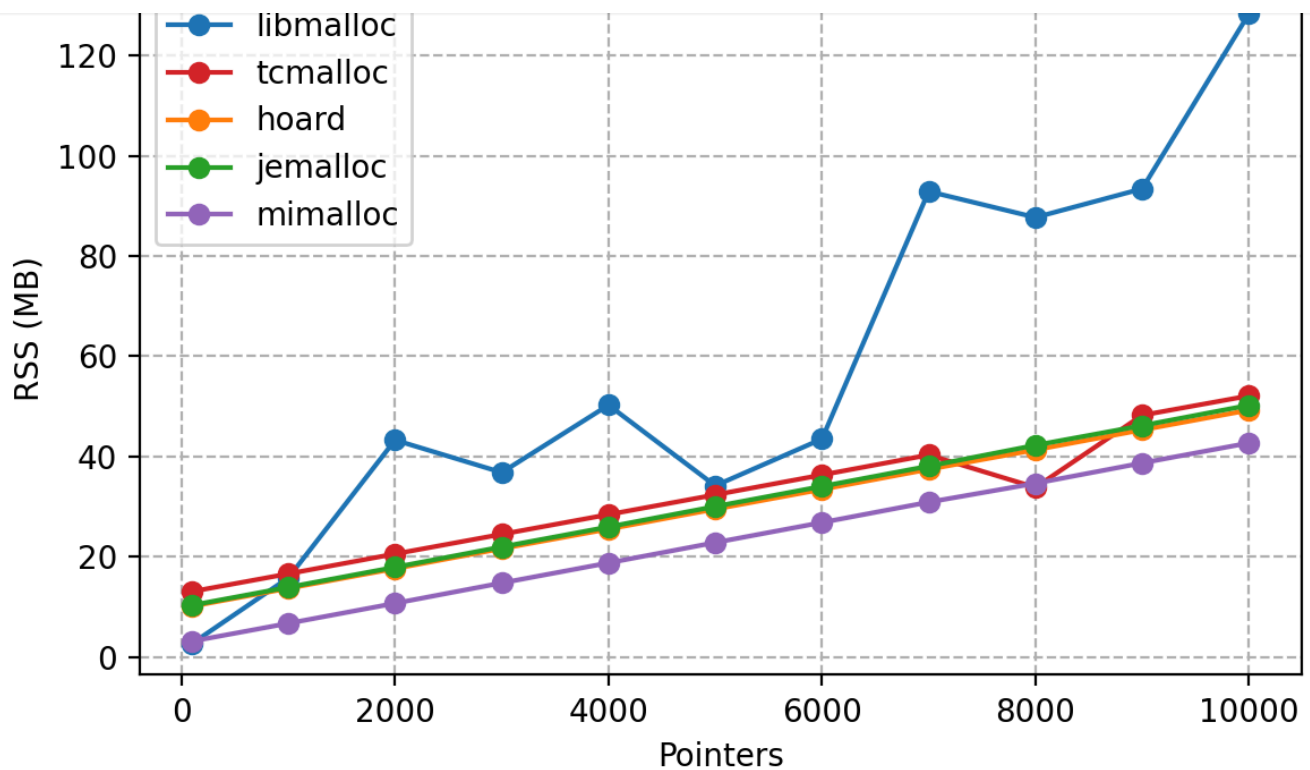
The following graph plots the RSS size of the program over time for different allocators, running for 1 seconds with 1000 pointers and an allocation size of 1KB per pointer in 1 thread:



First, we can see that all allocators except `libmalloc` reach a stable RSS size immediately after the first measurement. The increase from the starting size to the stable size corresponds to the total size of allocated memory ($1000 * 1\text{KB} = 1\text{MB}$). You can also note that `jemalloc` is the only allocator dropping back to the starting usage after stopping the threads.

The starting memory usage differs significantly between allocators. `libmalloc` and `mimalloc` both consume less than 4MB, while `hoard` and `jemalloc` consume ~9MB and `tcmalloc` is the most hungry one with ~13MB. However, we can see that `libmalloc` reaches a stable size of ~13MB after 5 seconds as well.

Next, let's investigate the RSS usage for an increasing allocation size. When looking at the stable RSS size (1 second before the end) for each allocator, using 1KB allocations with a varying number of pointers and 4 threads, we can see that `libmalloc` indeed behaves differently than all the other allocators.



While the RSS size for all other allocators increases linearly with the allocated memory, `libmalloc` appears to consume much more memory than being allocated. A similar issue has been observed with the `glibc` default allocator on Linux and RocksDB, where the RSS was 3x higher compared to `jemalloc` (see [Battle of the Mallocators](#) for more details).

Tooling and Usability

For most applications, using the default allocator with the default settings is good enough. For some applications, you might want to switch to a different allocator. However, there are also very specialized applications, that either have very specific performance requirements, or are memory constrained. For those, the default settings might not be the best. Additionally, you might want to debug your allocation workload, e.g. by collecting and inspecting allocation statistics. Let's look into the tooling and configuration options for each allocator.

`libmalloc`

`libmalloc` allows some debugging configuration via environment variables, but there is no programmatic API or compile-time options, and little to no documented tuning options.

`MallocStackLogging` is enabled, heap integrity checking via `MallocCheckHeapStart`, and a few other options.

`jemalloc`

`jemalloc` has three configuration mechanisms:

1. Environment variables. Via `MALLOC_CONF`, you can control nearly every aspect of the allocator. Example:
`MALLOC_CONF="prof:true,lg_prof_sample:19,prof_prefix:jeprof.out,narenas:4,dirty_decay_ms:5000"` will enable heap profiling, sample every 512KB (2^{19} B), write the profile to `jeprof.out`, use 4 arenas, and decay dirty pages after 5 seconds.
2. Programmatic API. You can use the `mallctl` interface to change the settings at runtime without restarting.
3. Compile-time options. Configuration can be baked via `--with-malloc-conf` or the `malloc_conf` global variable.

In terms of debugging and profiling, `jemalloc` has a wide variety of features. The main tool is `jeprof`, that analyzes heap dumps and generates flame graphs. By enabling the `prof_leak` option, allocations without matching `free` calls are reported.

```
MALLOC_CONF="prof:true,prof_prefix:jeprof.out,lg_prof_interval:5" \  
  build/malloc-post-rss-jemalloc 4 1000 1024 10  
jeprof --pdf build/malloc-post-rss-jemalloc jeprof.out.0
```

Note that in order to enable the profiling hooks, `jemalloc` needs to be configured with `--enable-prof`, which is not the case when installing it via homebrew. I was not able to compile it from source within a reasonable time frame, so I am not able to show the results here. But the features and presentation are very similar to what `tcmalloc` has to offer.

`tcmalloc`

`tcmalloc` also has three configuration mechanisms:

1. Environment variables. Via different `TCMALLOC_*` variables, you can configure things like release rates, size thresholds, limiting the heap size, etc.
2. Programmatic API. You can use the `MallocExtension` APIs to query and modify allocation parameters at runtime. Example: `MallocExtension::instance()->SetNumericProperty("tcmalloc.max_per_cpu_cache_size", 16777216);`

allocator.

In terms of tooling, `tcmalloc` is part of the Google Performance Tools (`gperftools`). The main relevant tool is `pprof`, which is similar to `jeprof` and used for analyzing heap profiles.

```
HEAPPROFILE=malloc-post-tcmalloc.hprof \
  build/malloc-post-rss-tcmalloc 4 1000 1024 1
```

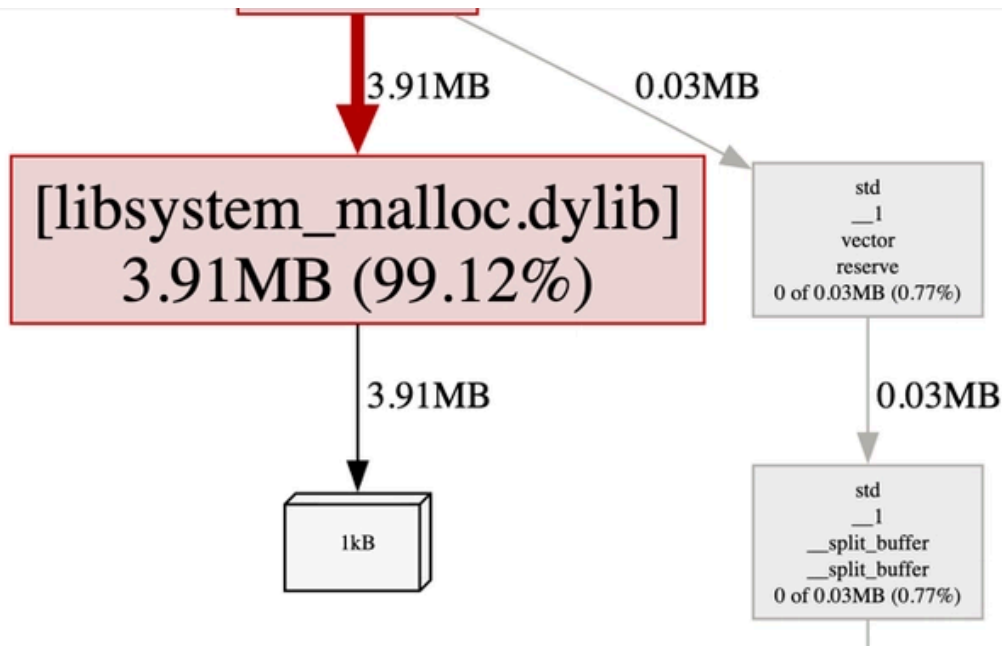
```
pprof -http=localhost:8080 build/malloc-post-rss-tcmalloc \
  malloc-post-tcmalloc.hprof.0001.heap
```

It has multiple output formats, including a comprehensive Web UI, that can show flame graphs, call graphs, or a cumulated view (top):

Flat	Flat%	Sum%	Cum	Cum%	Name
3.91MB	99.22%	99.22%	3.91MB	99.22%	[libsystem_malloc.dylib]
0.03MB	00.78%	99.99%	0.03MB	00.78%	std::__1::__libcpp_operator_new[abi:n
0	00.00%	99.99%	3.94MB	99.89%	worker_thread
0	00.00%	99.99%	0.03MB	00.78%	std::__1::vector::reserve
0	00.00%	99.99%	0.03MB	00.78%	std::__1::allocator::allocate[abi:ne1
0	00.00%	99.99%	3.94MB	99.89%	std::__1::__thread_proxy[abi:ne190102
0	00.00%	99.99%	3.94MB	99.89%	std::__1::__thread_execute[abi:ne1901
0	00.00%	99.99%	0.03MB	00.78%	std::__1::__split_buffer::__split_buf
0	00.00%	99.99%	0.03MB	00.78%	std::__1::__libcpp_allocate[abi:ne190
0	00.00%	99.99%	3.94MB	99.89%	std::__1::__invoke[abi:ne190102]
0	00.00%	99.99%	0.03MB	00.78%	std::__1::__allocate_at_least[abi:ne1
0	00.00%	99.99%	3.94MB	99.89%	[libsystem_pthread.dylib]



The graph view shows the allocation size (1KB), too:



`tcmalloc` also supports summary statistics using `MALLOCSTATS=1`:

```
MALLOCSTATS=1 build/malloc-post-rss-tcmalloc 4 1000 1024 1
```

```

-----
MALLOC:          20480 (    0.0 MiB) Bytes in use by application
MALLOC: +        3801088 (    3.6 MiB) Bytes in page heap freelist
MALLOC: +        372600 (    0.4 MiB) Bytes in central cache freelist
MALLOC: +       1048576 (    1.0 MiB) Bytes in transfer cache freelist
MALLOC: +          136 (    0.0 MiB) Bytes in thread cache freelists
MALLOC: +       2621504 (    2.5 MiB) Bytes in malloc metadata
MALLOC: -----
MALLOC: =         7864384 (    7.5 MiB) Actual memory used (physical + swap)
MALLOC: +           0 (    0.0 MiB) Bytes released to OS (aka unmapped)
MALLOC: -----
MALLOC: =         7864384 (    7.5 MiB) Virtual address space used
MALLOC:
MALLOC:          141          Spans in use
MALLOC:           1          Thread heaps in use
MALLOC:        8192          Tcmalloc page size
-----

```

mimalloc

Similar to `libmalloc`, `mimalloc` supports some basic environment variable configuration. In contrast to `libmalloc`, it does support some performance related configuration.

exhaustive tunability.

Tooling support for `mimalloc` is smaller compared to `jemalloc` and `tcmalloc`. It does not include a built-in sampling profiler, so you have to rely on external tools such as Valgrind. You can pass `MIMALLOC_SHOW_STATS` to get some basic statistics though.

```
MIMALLOC_SHOW_STATS=1 build/malloc-post-rss-mimalloc 4 1000 1024 1
```

```
heap stats:      peak      total      current      block      total#
reserved:      1.0 GiB      1.0 GiB      1.0 GiB
committed:    11.1 MiB      11.5 MiB      11.1 MiB
  reset:         0
  purged:         0
  touched:         0          0          0          ok
    pages:        68          68          0          ok
-abandoned:    13.6 Ki      17.3 Mi      0          ok
-reclaiam:         0
-reclaimf:     17.3 Mi
-reabandon:         0
  -waits:         0
  -extended:         0
  -retire:         0
  arenas:         1
  -rollback:         0
    mmap:         17
  commits:         0
  resets:         0
  purges:         0
  guarded:         0
  threads:         4          4          0          ok
  searches:      1.0 avg
numa nodes:         1
  elapsed:       2.011 s
process: user: 4.010 s, system: 0.030 s, faults: 94, rss: 6.7 MiB, commit:
```

hoard

hoard does not appear to have any configuration options or specific profiling tools.

Maintenance and Security

Apple's `libmalloc` has sophisticated security features such as `kalloc_type` and `xzone malloc`. While it has the highest CVE count¹, I believe this is due to extensive security

`jemalloc` and `tcmalloc` prioritize performance over security hardening, with minimal built-in protections. They have a handful of historical CVEs (`jemalloc`², `tcmalloc`³) reported, which are all patched in recent versions.

`mimalloc` offers the most comprehensive configurable security mode with guard pages, encrypted free lists, and randomization at ~10% performance cost. It has no core CVEs reported, only one minor advisory for the rust crate.⁴

`hoard` has the weakest security posture with documented overflow vulnerabilities such as multiple overflow vulnerabilities and no hardening features.

Summary and Conclusion

Based on the findings and my limited experience with using the allocators when developing this post, I came up with the following comparison table. I will leave out `hoard`, because I don't think it's a practical choice for any real world application.

	<code>libmalloc</code>	<code>tcmalloc</code>	<code>jemalloc</code>	<code>mimalloc</code>
Throughput	★★★★☆☆	★★★★★☆☆	★★★★☆☆☆☆	★★★★☆☆☆☆
Latency	★★★★★★★★	★★★☆☆☆☆	★★★★★☆☆	★☆☆☆☆☆☆
Memory Overhead	★☆☆☆☆	★★★★★★★★	★★★★★☆☆	★★★★★☆☆
Tooling and Usability	★★★★☆☆	★★★★★★★★	★★★★★☆☆	★★★★☆☆☆☆
Maintenance and Security	★★★★★★★★	★★★★★☆☆	★★★★☆☆☆☆	★★★★★★★★
Overall	16/25 ★	20/25 ★	18/25 ★	15/25 ★

On all Apple operating systems, `libmalloc` is the default choice. The main focus is on security, with decent performance for most workloads. Given the high memory overhead however, it might not be a good fit for performance critical applications with high allocation rates.

`tcmalloc` and `jemalloc` have the most stable performance characteristics across different allocation sizes and number of threads. The throughput remains reasonably

very large allocations. Among the ones I tested, those would be my preferred choice for applications with high performance requirements. They also have extensive configuration options that allows workload specific tuning. Note, however, that `jemalloc` appears to [somewhat dead](#) as of 2025, so I'd rather go with `tcmalloc` for any new project.

`mimalloc` works well for smaller allocations, but suffers in both latency and throughput for larger allocations. The advanced security features might be a unique selling point for some users though.

While `hoard` shines in certain areas, it does not appear to be a good choice, as it does not have steady performance characteristics across different allocation sizes and number of threads. It has severe security flaws and is not actively maintained.

Did you ever swap out the default allocator in your application? What was your experience? Let me know in the comments below!

If you liked this post, you can [support me on ko-fi](#).

1. [CVE-2015-5889, CVE-2018-4433, CVE-2023-32428](#) ↩
2. [CVE-2007-6754, CVE-2006-7252](#) ↩
3. [CVE-2005-4895](#) ↩
4. [RUSTSEC-2022-0094](#) ↩



MongoDB Atlas runs apps anywhere.

[Start Building](#)

Build seamlessly, securely, and flexibly with MongoDB Atlas. Try free.

MongoDB Atlas lets you build and run modern apps in 125+ regions across AWS, Azure, and Google Cloud. Multi-cloud clusters distribute data seamlessly and auto-failover between providers for high availability and flexibility. Start free!

[Learn More](#)

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)

**Frank Rosner**

My professional interests are cloud and big data technologies, machine learning, and software development. I like to read source code and research papers to understand how stuff works.

Pronoun: He

WORK

Senior Cloud Software Engineer @ ClickHouse

JOINED

Oct 9, 2017

More from [Frank Rosner](#)

Comparing OpenBLAS and Accelerate on Apple Silicon for BLAS Routines

[#cpp](#) [#simd](#) [#matrix](#) [#cpu](#)

Writing My Own Dynamic Memory Management

[#memory](#) [#algorithms](#) [#datastructures](#) [#c](#)

Hit Me Baby One More Time - What Are Cache Hits and Why Should You Care?

[#programming](#) [#performance](#) [#scala](#) [#computerscience](#)



Explored Web3 but not sure where to start?

100 Days of Solana is a hands-on challenge series that takes you from curiosity to capability. By the end, you won't just understand Solana, you'll be ready to build something useful with it.

[Start here](#)